

Decisões de Arquitetura

Cada decisão técnica no formato contexto → decisão → alternativa descartada → consequência, reconstruído a partir do código e do README deste repositório. Quinze ADRs, o fluxo ponta a ponta, perguntas frequentes e os riscos conhecidos.

ÍNDICE		
00	Resumo executivo	ADR
01	Fluxo ponta a ponta	ADR
ADR	Domínio puro + Protocols	ADR
ADR	RAG em vez de fine-tuning	ADR
ADR	pgvector no mesmo Postgres	ADR
ADR	LiteLLM como gateway único	ADR
ADR	Ollama local (qwen2.5:3b)	ADR
ADR	Loop de ferramentas = resposta final	02
ADR	Isolamento multi-tenant	ADR

Resumo executivo

PROPOSTA
Um copiloto de atendimento para e-commerce que responde com base em documentos internos (RAG com pgvector) e executa uma ação real via ferramenta (consulta de pedido), usando um agente LangGraph com tool calling. É multi-tenant: cada loja só enxerga os próprios documentos e pedidos. Escopo pequeno, execução de produção — gateway único de modelo, prompt versionado, guardrails nas duas bordas e observabilidade em vez de testes de igualdade exata sobre texto gerado.

Fluxo ponta a ponta

POST /chat + Authorization: Bearer chave + chave autenticada contra tenants.chave_api_hash (401 se inválida/ausente) → guardrail de entrada (Pydantic: 1-4000 chars, strip, não-vazio) → agente LangGraph agente -> ModelGateway.complete_with_tools (stream=False) tool_call? sim + executar buscar_documentos ou consultar_pedido (tenant_id fixado por closure, nunca vindo do modelo) → resultado sanitizado volta ao histórico + agente decide de novo (até MAX_ITERACOES = 4) não → segue por passo final ModelGateway.stream_completion (stream=True, resposta final) → mascarar_stream_pii (libera só até o último espaço em branco) → SSE (data: ...pedaç... até data: [DONE]) + cliente cada chamada de modelo loga: tenant_id, prompt_id, tokens, custo_usd, latencia_segundos ao fim do stream, loga a interação inteira — mensagem e resposta já mascaradas por mascarar_pii

ADR-001 Domínio puro + Protocols como portas	DEFINIDA
CONTEXTO A regra de negócio (decidir status, montar contexto, mascarar PII) precisa ser testável sem Postgres, Ollama ou FastAPI reais, e o provider de LLM/banco pode mudar sem reescrever o domínio. ALTERNATIVA DESCARTADA Repositório e chamada ao LiteLLM direto dentro da rota — mais rápido de escrever, mas testar a regra de decisão do agente ou o mascaramento de PII exigiria subir Postgres/Ollama reais a cada teste.	DECISÃO A suíte padrão (tests/unit + tests/feature) roda com duplês em memória — nunca toca infraestrutura real, só tests/evals (fora de testpaths) precisa do Postgres+Ollama de pé. CONSEQUÊNCIA A suíte padrão (tests/unit + tests/feature) roda com duplês em memória — nunca toca infraestrutura real, só tests/evals (fora de testpaths) precisa do Postgres+Ollama de pé.

Separo domínio, portas e adapters porque quero testar a decisão do agente e os guardrails sem depender de Postgres, Ollama ou rede — a suíte roda em segundos e o provider de LLM/banco vira detalhe de infraestrutura, não do teste.

ADR-002 RAG em vez de fine-tuning	DEFINIDA
CONTEXTO As políticas da loja (prazo de devolução, frete, garantia, pagamento) mudam com frequência e variam por tenant. ALTERNATIVA DESCARTADA Fine-tuning do modelo nas políticas — atualizar uma política viraria retreinar o modelo, e a resposta deixaria de ser auditável (não dá pra apontar qual trecho a embasou).	DECISÃO RAG: os documentos ficam em data/sample_docs/tenant/; são ingeridos com embedding e recuperados por similaridade a cada pergunta. CONSEQUÊNCIA Atualizar uma política é reingier um documento (scripts/ingest_documents.py), sem retreinar nada; toda resposta é rastreável até o título do documento recuperado. O sistema já sai preparado pra um futuro modelo fine-tunado por loja, sem precisar dele agora (ADR-015).

Fine-tuning faria sentido pra ensinar um estilo de escrita, não fatos que mudam toda hora — RAG é reingestão, não retreino, e ainda sobra uma trilha auditável do que embasou a resposta.

ADR-003 pgvector em vez de vector DB dedicado	DEFINIDA
CONTEXTO Dados relacionais (pedidos) e vetoriais (documentos) coexistem no mesmo domínio de problema. ALTERNATIVA DESCARTADA Pinecone/Weaviate/Qdrant dedicado — complexidade sem benefício nesse volume: mais uma engine, mais uma credencial, mais um ponto de falha, sem ganho de performance perceptível.	DECISÃO Um único Postgres (pgvector/pgvector:pg16) com a extensão vector; documentos.embedding VECTOR(384) com índice hnsw (vector_cosine_ops), junto da tabela pedidos. CONSEQUÊNCIA Um único backup, uma única transação possível entre dado relacional e vetorial, um único healthcheck no docker-compose.yml.

Pra esse volume, um Postgres a mais é complexidade sem benefício — dado relacional e vetorial vivem na mesma engine, mesmo backup, mesma transação.

ADR-004 LiteLLM como gateway único de modelo	DEFINIDA
CONTEXTO Trocar de provider de LLM (Ollama local → hospedado) não deveria tocar código de negócio. ALTERNATIVA DESCARTADA Importar o client do provider (openai, anthropic, ollama) direto onde for usado — funciona, mas espalha a decisão de provider pelo código e cada troca vira um refator.	DECISÃO Nenhum SDK de provider é importado fora de ModelGateway (app/orchestration/gateway/model_gateway.py); toda chamada passa por litellm.acompletion / litellm.aembedding. CONSEQUÊNCIA Trocar de Ollama para qualquer provider suportado pelo LiteLLM é mudar LLM_MODEL no .env, não código — confirmado pelo próprio api_base(), que só preenche a URL do Ollama quando llm_provider == 'ollama'.

Nenhum SDK de provider aparece fora do gateway — trocar de Ollama pra OpenAI ou Anthropic é variável de ambiente, não deploy de código novo.

ADR-005 Ollama local (qwen2.5:3b) como provider padrão	DEFINIDA
CONTEXTO Projeto de portfólio: custo de API de LLM hospedado não se justifica, mas tool calling confiável em modelo pequeno não é garantido. ALTERNATIVA TESTADA E DESCARTADA llama3.2:1b: numa pergunta que precisava da ferramenta, respondeu com um JSON solto como texto; numa saudação simples, inventou uma chamada de ferramenta com dado fictício. qwen2.5:3b decide corretamente nos dois casos.	DECISÃO qwen2.5:3b via Ollama local, com ollama_chat/ como prefixo do modelo no LiteLLM — não ollama/ — porque é o prefixo que habilita tool calling estruturado nessa integração. CONSEQUÊNCIA Custo zero, mas CPU-bound: cada resposta de /chat leva dezenas de segundos contra um modelo de 3B sem GPU (números reais no Dashboard de Execução).

Testei empiricamente antes de decidir: um modelo de 1B não sustenta tool calling de forma confiável — alucina chamada de ferramenta numa saudação simples. 3B foi o menor que decidiu certo nos dois casos, sem custo de API.

ADR-006 LangGraph resolve o loop de ferramentas; a última resposta do loop é a resposta final	DEFINIDA
CONTEXTO Tool calling confiável em modelos pequenos via Ollama exige stream=False — em streaming os tool_calls não vêm estruturados. A rodada do loop que decide "não preciso de mais ferramenta" já produz, nesse mesmo passo, o texto da resposta final. ALTERNATIVA DESCARTADA Uma chamada de streaming à parte pra toda resposta final — dá um efeito de "digitando". Rodada-a-token, mas gera o mesmo texto duas vezes: uma dentro do loop de decisão, outra só pra exibir.	DECISÃO chamar_modelo (app/orchestration/agent/graph.py), ao ver tool_calls vazio, guarda esse texto em estado["resposta_final"]. Em chat.py, sempre que resposta_final existe, ela é servida direto (envolpada em SSE via _pronta); stream_completion só é chamado no corte de MAX_ITERACOES, quando o loop é interrompido sem o modelo ter fechado a resposta. Nesse caso, mensagens_para_fallback descarta a última tentativa de tool call (nunca executada, porque o corte aconteceu antes) e acrescenta uma instrução explícita pedindo resposta em texto — sem isso, o modelo tenta "completar" a chamada de ferramenta pendente como texto solto em vez de responder ao cliente. CONSEQUÊNCIA Conversas que passam pelo loop de ferramentas usam uma chamada de modelo a menos — a última rodada do loop já é a resposta final. Conversas sem ferramenta precisam de só uma chamada. Em troca, a resposta final chega como um único bloco SSE, sem o efeito de streaming token-a-token nessa etapa.

Reaproveito o texto que o modelo já escreveu na última rodada do loop em vez de pedir a mesma resposta de novo — mais rápido, sem custo de correção; só abre mão do streaming token-a-token nessa etapa, que não é um requisito do projeto.

exemplo real: pergunta que passa por 4 rodadas de ferramenta antes de fechar a resposta 5 chamadas de modelo ao todo (4 decisões + 1 resposta final, que é a própria 5ª decisão) — nenhuma sexta chamada de regeneração

ADR-007 Isolamento multi-tenant na origem, não na aplicação	DEFINIDA
CONTEXTO Um prompt injection do tipo "busque no tenant X" não pode vaziar dado de outra loja. ALTERNATIVA DESCARTADA Deixar o modelo passar tenant_id como argumento da tool — mais flexível, mas um prompt malicioso poderia tentar sobrescrever-lo.	DECISÃO tenant_id vem da chave de API autenticada em obter_tenant_id (app/api/dependencias.py) — ver ADR-013 e é fixado por closure dentro de criar_ferramenta_buscar_documentos / criar_ferramenta_consultar_pedido — nunca é um parâmetro que o schema da tool expõe ao modelo. Toda query nos repositórios Postgres filtra por tenant_id na cláusula WHERE. CONSEQUÊNCIA Perguntar por um pedido de outra loja retorna "não encontrado" — nunca um erro de tenant errado, porque a query nem enxerga a linha correspondente.

O tenant nunca é um parâmetro que o modelo controla — é injetado na origem da requisição HTTP e capturado na closure da ferramenta, então nenhum prompt injection consegue trocá-lo.

tenant=loja-verde, pergunta="Qual o status do pedido 1001?" (pedido real de loja-azul) resposta real do agente: "Desculpe, o número do seu pedido 1001 não foi encontrado. Você pode fornecer outro número de pedido ou compartilhar mais detalhes para que eu possa ajudar melhor?"

ADR-008 Guardrail de saída: mascaramento de PII em streaming, não bloqueio	DEFINIDA
CONTEXTO Bufferizar a resposta inteira antes de validar materia o streaming; mas cortar um pedaço no meio de um CPF ou e-mail deixaria o padrão passar sem ser mascarado. ALTERNATIVA DESCARTADA Bufferizar a resposta inteira e só então mascarar — eliminaria o risco de corte no meio de um padrão, mas anula o streaming real que o ADR-006 foi buscar.	DECISÃO mascarar_stream_pii (app/domain/guardrails/pii.py) acumula um buffer e só libera (yield) texto até o último espaço em branco encontrado — o resto fica retido até o próximo pedaço. Os mesmos 4 padrões (CPF, e-mail, cartão, telefone) são aplicados tanto na resposta ao cliente quanto no log da interação. CPF, cartão e telefone não usam \b na borda do padrão — letra e dígito não formam fronteira de palavra pro regex, e uma PII colada a uma palavra ("cp123.456.789-00") precisa ser capturada do mesmo jeito; e-mail mantém \b, já coberto pela própria estrutura do @. CONSEQUÊNCIA Nenhum PII fica cortado ao meio entre dois pedaços do SSE; em compensação, o cliente só vê cada palavra depois que o próximo pedaço em branco chega.

Não bufferizo a resposta inteira pra não matar o streaming — só seguro o texto até o último espaço, tempo suficiente pra nenhum CPF ou e-mail ficar cortado ao meio entre dois pedaços. \b nos três padrões numéricos deixaria escapar PII colada a uma letra — por isso eles não usam.

tenant=loja-azul, pergunta="Meu email e cliente.teste@exemplo.com, pode confirmar o pedido 18922?" log real (interacao_chat concluída): mensagem_usuario: "Meu email e [E-MAIL OCULTADO], pode confirmar o pedido 18922?"

ADR-009 Conteúdo de fora do modelo é input não confiável	DEFINIDA
CONTEXTO Um documento de política poderia conter, por acidente ou ataque, um marcador de instrução que o modelo interpretasse como comando. ALTERNATIVA DESCARTADA Confiar no conteúdo dos documentos por serem "internos" — frágil: a mesma pipeline de ingestão poderia um dia indexar conteúdo gerado por terceiros.	DECISÃO sanitizar_conteudo_externo (app/domain/guardrails/sanitizacao.py) trunca o texto recuperado em 2000 caracteres e substitui a primeira ocorrência de cada marcador suspeito (system:, assistant:, ##, "ignore as instruções anteriores") por [trecho removido], antes do resultado da tool voltar ao histórico da conversa. CONSEQUÊNCIA O resultado de buscar_documentos recebe o mesmo tratamento, que a entrada do usuário — nenhum dos dois é implicitamente confiável dentro do histórico da conversa.

Trato o retorno da busca de documentos como eu trato a entrada do usuário: não confio, sanitizo — um documento não deveria conseguir instruir o agente só por estar no contexto.

ADR-010 Observabilidade com logging JSON estruturado, sem Langfuse	DEFINIDA
CONTEXTO Cada chamada de modelo precisa ser auditável (quanto custou, quanto demorou, pra qual tenant) sem depender de um serviço externo. ALTERNATIVA DESCARTADA Langfuse — daria uma UI de trace visual, mas é dependência externa a mais pra rodar o projeto.	DECISÃO configurar_logging() (app/config/logging.py) instala um FormattadorJSON no stdout; registrar_chamada (ModelGateway) loga tenant_id, prompt_id, model, latencia_segundos, e — só quando a resposta não é streaming — custo_usd (via litellm.completion_cost, best-effort) e tokens_entrada / tokens_saida (usage não vem exposto em streaming). CONSEQUÊNCIA Zero UI de trace visual; trocar pra Langfuse depois é adicionar um cliente no ModelGateway — a interface pública não muda.

Não preciso de Langfuse pra logar tenant, custo e latência de cada chamada — um logger JSON no stdout resolve sem dependência externa, e trocar depois é isolado no gateway.

log real (chamada_modelo concluída, tenant=loja-azul, pergunta sobre devolução): {"tenant_id": "loja-azul", "prompt_id": "chat.agente@1", "model": "ollama_chat/qwen2.5:3b", "latencia_segundos": 26.712, "custo_usd": 0.0, "tokens_entrada": 413, "tokens_saida": 23}

ADR-011 Qualidade avaliada com recall@k + limiar, não igualdade exata	DEFINIDA
CONTEXTO Testar "a resposta é exatamente X" não faz sentido pra geração de texto — o mesmo fato pode ser dito de formas diferentes. ALTERNATIVA DESCARTADA Comparar a resposta final do agente char-a-char com um gabarito — frágil a qualquer variação de fraseado do modelo, sem medir o que realmente importa (a busca recuperou o documento certo?).	DECISÃO tests/evals/test_eval_rag.py roda os 10 casos de dataset_rag.py (5 por tenant) contra o Postgres+pgvector real, verifica se o título esperado aparece no top-3 recuperado e falha se o recall cair abaixo de LIMIA_RECALL = 0.7. CONSEQUÊNCIA Fica fora da suíte padrão — exige infraestrutura real de pé mas mais exatamente a camada que pode regredir silenciosamente: a qualidade da recuperação vetorial.

Testar igualdade exata contra texto gerado é frágil e não mede o que importa — meço se a busca recuperou o documento certo, que é a camada onde uma regressão de verdade aconteceria.

medido contra o Postgres+pgvector real: recall@3 = 10/10 = 1.00 — os 10 casos (5 loja-azul, 5 loja-verde) recuperam o documento esperado, sempre na primeira posição

ADR-012 Rate limiting por (IP, tenant) — não documentado no README, encontrando no código	CÓDIGO-DERIVADO
CONTEXTO POST /chat é o único endpoint que dispara processamento caro (embedding + inferência); sem limite, um único cliente (ou um único tenant atrás de um NAT) conseguiria monopolizar o modelo local pra todos os usuários. ALTERNATIVA DESCARTADA Só IP — múltiplos tenants atrás do mesmo NAT dividiriam o mesmo teto de 20/min; um tenant sozinho conseguiria estourar a cota de todos os outros. Só tenant (sem IP) também foi descartado: um único tenant com tráfego legítimo de vários clientes/dispositivos perderia a cota rápido demais.	DECISÃO obter_chave_limite (app/api/limiter.py) combina IP e tenant: f'ip:{tenant_id}'. O tenant já foi autenticado e resolvido em obter_tenant_id (ADR-013), que grava o resultado em request.state.tenant_id, de onde o rate limiter (ADR-012) e o restante da rota leem sem repetir a consulta. CONSEQUÊNCIA Revoar uma loja é DELETE /rotacionar uma linha na tabela, sem redeção. Não existe endpoint de gestão de chaves — rotação é manual, via scripts/seed_tenants.py ou acesso direto ao banco.

Chave de API é a solução mais simples que resolve o problema real (tenant autodeclarado) sem a complexidade de um emissor de JWT — e hash em vez de texto puro é o mesmo cuidado que se tem com senha, aplicado à chave.

contra o banco real: sem Authorization = 401 · Bearer chave-fake = 401 (chave de API inválida) · Bearer dev-loja-azul = 200, tenant resolvido corretamente

ADR-014 Validação de dimensão do embedding antes do insert	DEFINIDA
CONTEXTO documentos.embedding é VECTOR(384) fixo no schema; trocar LLM_EMBEDDING_MODEL por um modelo de dimensão diferente quebrava a inserção só com o erro genérico do Postgres. ALTERNATIVA DESCARTADA Deixar o Postgres rejeitar — funciona, mas a mensagem de erro do driver não aponta pra causa raiz (toca de modelo de embedding), só pro sintoma (tamanho de vetor incompatível).	DECISÃO deitar_para_sql (documento_repository.py) valida len(embedding) == DIMENSAO_EMBEDDING (384) antes de montar o SQL — usado tanto por inserir quanto por buscar_similares — e levanta ValueError apontando a causa (dimensão recebida vs. esperada) e o culpado provável (LLM_EMBEDDING_MODEL). CONSEQUÊNCIA Erro acontece na camada de aplicação, com mensagem acionável, antes de qualquer round-trip ao banco.

Uma constante e uma checagem resolvem — não precisa de mais que isso pra transformar um erro genérico de driver num erro que já aponta a causa.

ADR-015 Preparo estrutural para fine-tuning por tenant, sem fazer fine-tuning	DEFINIDA
CONTEXTO RAG foi escolhido em vez de fine-tuning como estratégia de conteúdo (ADR-002), porque políticas mudam com frequência. Isso não impede que uma loja específica, no futuro, use um modelo próprio (fine-tunado por seu domínio ou tom de voz) — mas o sistema precisa ter onde encaixar isso sem re-arquitetar nada. ALTERNATIVA DESCARTADA Fazer o fine-tuning agora — fora de escopo: o ADR-002 já rejeitou fine-tuning como estratégia de conteúdo. O que existe aqui é só o encaixe estrutural (tag + seletor de modelo), não um treino.	DECISÃO Toda chamada ao LiteLLM (complete_with_tools, stream_completion, embed) passa metadata= {"tenant_id": ..., "prompt_id": ...} — cada interação já sai etiquetada por loja, pronta pra segmentar um dataset de treino por tenant no futuro. Separadamente, tenants_modelo_override (nullable) permite que uma loja use um modelo litellm diferente do LLM_MODEL global; obter_tenant_id resolve esse valor na mesma consulta que autentica a chave (TenantAutenticacao) e repassa via request.state e o estado do grafo (modelo) até o gateway. CONSEQUÊNCIA Modelo_override começa NULL pra todo tenant, então nenhum comportamento muda por padrão — cai no LLM_MODEL global. Quando uma loja tiver um modelo fine-tunado, plugar é um UPDATE numa linha da tabela, sem mudança de código.

Preparo sem custo pra quem não usa: a tag de tenant nas chamadas já organiza dados por loja, e o campo de modelo por tenant é só uma coluna nullable — zero impacto em quem nunca vai precisar.

Perguntas frequentes

Por que Protocol e não uma classe abstrata (ABC) nas portas dos repositórios?

Protocol é tipagem estrutural: um fake de teste não precisa herdar de DocumentoRepositoryInterface, só ter os métodos certos (buscar_similares, inserir). Menos acoplamento entre o código de teste e o de produção.

Por que dataclass no domínio (Documento, Pedido) e Pydantic só no DTO de entrada?

O domínio não depende de nenhum framework, nem Pydantic — são dataclasses congeladas da stdlib. Pydantic entra só onde a borda HTTP precisa validar um payload externo (MensagemChatEntrada): é fronteira de I/O, não regra de negócio.

Por que o grafo (GRAFO) é compilado uma única vez, na importação do módulo?

StateGraph.compile() não é barato e a topologia (dois nós, uma areia condicional) não muda entre requisições — só o estado (gateway, ferramentas, tenant_id) varia, e esse já entra via ainvoke, não via reconstrução do grafo.

O que acontece se o Ollama cair no meio de uma resposta?

Não há handler dedicado: main.py só registra um exception handler para RateLimitExceeded. Uma falha do LiteLLM em complete_with_tools ou stream_completion sobe como exceção não tratada — a API responde com o 500 padrão do FastAPI/Starlette, sem retry nem fallback de provider.

Decisões reconstruídas a partir do código e do README — não substitui o histórico real de decisão do autor, só documenta o estado atual no formato ADR.